

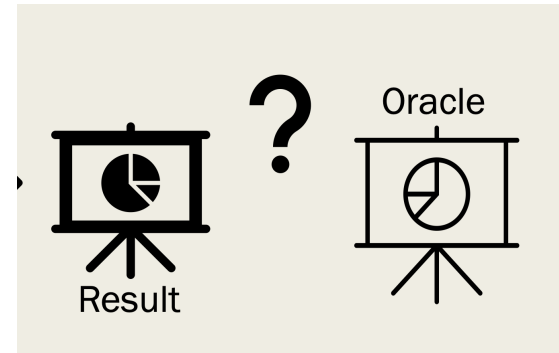
COMS 4170
Software Testing
Spring 2026

Metamorphic
Testing



1

What is the Correct Answer?



2

Trivial Oracles

Program crashes

Core dump

Segmentation error

Overflow

Program hangs

3

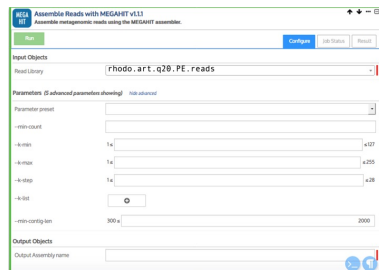
Trivial Oracles

- Good when we don't have a known result
- **Weakest oracle** since it only shows that the program fails/not that the result is incorrect
- Exact oracles are easy to compute in some programs

```
def classify_triangle(a, b, c):  
    # Sort the sides so that a <= b <= c  
    tmp = a  
    a = tmp # result should be a <= b  
    b = tmp  
  
    if a > b:  
        tmp = a  
        a = b  
        b = tmp  
  
    if a > c:  
        tmp = a  
        a = c  
        c = tmp  
  
    if b > c:  
        tmp = b  
        b = c  
        c = tmp  
  
    if a + b <= c:  
        return TriangleType.INVALID  
    elif a == b and b == c:  
        return TriangleType.EQUILATERAL  
    elif a == b or b == c:  
        return TriangleType.ISOSCELES  
    else:  
        return TriangleType.SCALENE
```

4

Harder Oracles



5

Making Oracles Hard

- Results may **differ by small epsilon**s (due to rounding)
- Expected result **may not be computable** without program
- May have **time series** results
- Takes a long time to manually compute each oracle (even when we can)
- Programs may be **stochastic** (or flaky)

6

Examples

Python docs

Note: The behavior of `round()` for floats can be surprising: for example, `round(2.675, 2)` gives 2.67 instead of the expected 2.68. This is not a bug: it's a result of the fact that most decimal fractions can't be represented exactly as a float. See [Floating Point Arithmetic: Issues and Limitations](#) for more information.

Same growth values?

Expected: 0.35695124
Observed: 0.35695122

Correct hits?

Descriptions	Graphic Summary	Alignments	Taxonomy
Sequences producing significant alignments			
select all 101 sequences selected			
Descriptions	Score	Expect	Ident
Bacillus subtilis strain DSM 9420, complete genome	1000	100%	0.0
Bacillus subtilis strain DSM 9420, complete genome	1000	100%	0.0
Bacillus subtilis strain DSM 9420, complete genome	1000	100%	0.0
Bacillus subtilis strain DSM 9420, complete genome	1000	100%	0.0

7

Some Techniques



Differential testing



Metamorphic testing

8

Differential Testing



Differential testing

*Run same tests using
different programs
that have the same
functionality*

9

Example from Biology

Trait	Tool A	Tool B	Tool C	Tool D
Metabolites	885	885	884	885
Reactions	1096	1229	1097	1229
Genes	885	900	900	900
Biomass	2.3565	0.0	0.5	0.0

– Models should produce:

- Same number of reactions, genes, etc.

Alexis L. Marsh, Myra B Cohen, and Robert W Cottingham. 2023. CORE: Comparable, Open, Reliable, Extensible Software: An Experience Report of Four Flux Balance Analysis Tools. In Proceedings of the 14th ACM International Conference on Bioinformatics, Computational Biology, and Health Informatics (BCB '23)

10

Metamorphic Testing



Metamorphic testing

Define relations on sets of tests:

e.g. (subtraction)

(1) $A - B = C$

Create A' (greater than A)

*(2)
 $A' - B = C'$ means C' is greater than C*

11

Many Types
of Programs
May Be
“Untestable”



Machine learning



Scientific applications

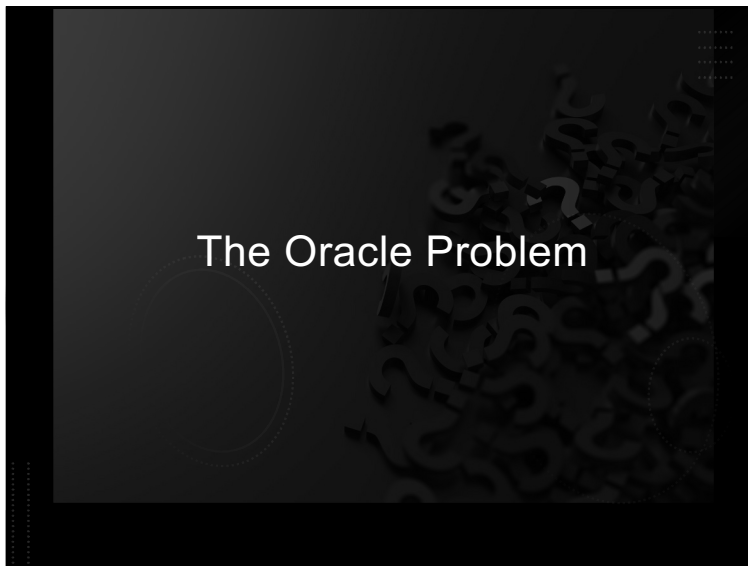


Databases



*How can we test if we
don't know the correct
answer?*

12



13

FEATURE: AUTOMATED SOFTWARE TESTING

Metamorphic Testing: Testing the Untestable

Sergio Segura, University of Seville

and the programs suffering from are often referred to as *nontestable* (or *untestable*).¹⁻³

Metamorphic testing (MT) is an effective technique for alleviating the oracle problem and, thus, for testing untestable programs where, as in the previous example, failures are not revealed by checking individual outputs but by checking the expected relationships among multiple executions of the

Reading

Segura, S., Towey, D., Zhou, Z.Q., Chen, T.Y., Testing: Testing the Untestable, *IEEE Software*, April, 2020

14

Metamorphic Testing: A Review of Challenges and Opportunities

TSONG YUEH CHEN and FEI-CHING KUO, Swinburne University of Technology
 HUAI LIU, Victoria University
 PAK-LOK POON, RMIT University
 DAVE TOWEY, University of Nottingham Ningbo China
 T. H. TSE, The University of Hong Kong
 ZHI QUAN ZHOU, University of Wollongong

Metamorphic testing is an approach to both test case generation and test result verification. A central element is a set of metamorphic relations, which are necessary properties of the target function or algorithm in relation to multiple inputs and their expected outputs. Since its first publication, we have witnessed a rapidly increasing body of work examining metamorphic testing from various perspectives, including metamorphic

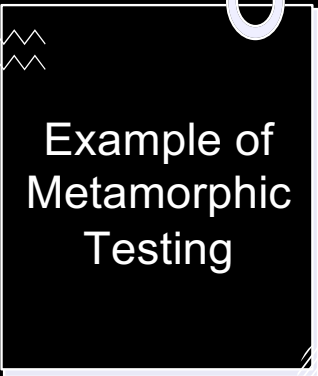
Optional Reading

- Chen, Kuo, Liu, Poon, Towey, Tse, Zhou

15

Where has it been Successfully Applied?

16



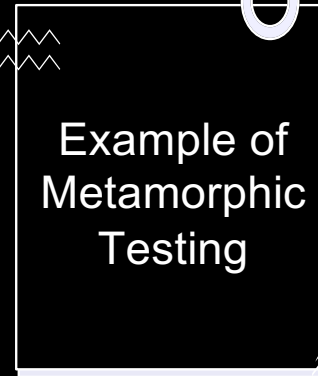
Example of Metamorphic Testing

Database query:

>> Count all purchases greater than \$100
>> 3,400

How do we know if this is correct?

17



Example of Metamorphic Testing

2nd Database query:

>> Count all purchases equal to, or greater than \$100
>> 3,300

3,300 is < 3,400 --> indicates a fault

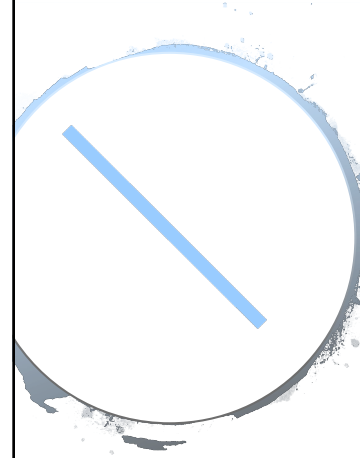
18



Shortest Path

- Suppose that an algorithm f computes the shortest path for an undirected graph G , and a program P implements f
- For any two vertices a and b in a large G , it may be very difficult to verify whether $P(G,a,b)$ is really the shortest path between a and b .

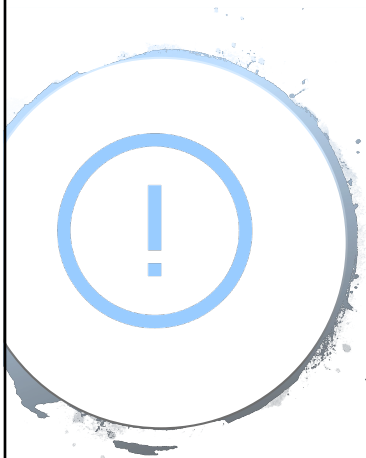
19



Shortest Path

- Could compute "all possible paths" – but this is infeasible
- Instead we can make use of some properties to partially verify the result
- A Metamorphic Relationship (MR) can be derived from the following property:
 - if the vertices a and b are swapped, the length of the shortest path will remain unchanged, that is, $|f(G,b,a)| = |f(G,a,b)|$.

20



How Can we Implement This?

- We **need two tests** (not one)
 - One with the source test case (G,a,b)
 - One other with the follow-up test case (G,b,a)
- We now check whether the relation $|P(G,b,a)| = |P(G,a,b)|$ is satisfied or violated. If a **violation is detected**, we can then say **that P is faulty**

21

Metamorphic Relations

-  In metamorphic testing we do not know the exact answer
-  Instead, we compare answers from more than one test
-  Our oracle is the relationship between those tests

22

Example

- Program **merge(L₁,L₂)**
 - Merges two lists into a single ordered list without duplicate elements
 - Deciding correctness for non-trivial program is hard

23

Example

- **What we do know:**
 - If we perform **merge(L₁, L₂)** and then perform **merge (L₂,L₁)** the order of merging should not change the outcome

Metamorphic relationship:
 $\text{merge}(L_1, L_2) = \text{merge}(L_2, L_1)$

Merge(L1,L2) → source test case
 Merge(L2,L1) → follow up test case

24

Example

- We can instantiate this relationship using many different inputs:

`merge([a,d],[k,t])=merge([k,t],[a,d])`

25

Test Generation

- Use the **metamorphic relationship** to generate many different test cases
- e.g. **many randomly generated lists** can be tried until one fails or some time out occurs

26

Simple Functional Example

- Subtraction: $A - B = C$
- Source Test: $C = A - B$
- Follow on Test
 - $C' = A - B'$ where $B' < B$We know that $C' > C$

27

Case Study

[booking.com](https://www.booking.com)

Website to [find potential lodgings](#) based on a set of criteria

Search for an accommodation in Rome with 7,378 results

- Is this correct?
- Is anything missing?
- Does anything in the result set fail the selected criteria?

28

Case Study

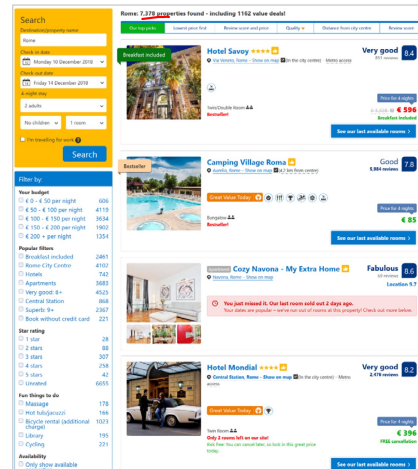


Figure 1. Booking.com search interface.

Metamorphic Relations

MR₁: Perform search, then perform a search with a budget filter (US \$100-150)
Result of second should be subset of first

MR₂: Perform search for “1 star” hotels.
Then repeat 4 times changing to 2, 3, 4, and 5 stars
Results of all 5 should be disjoint

MR₃: Perform a search. Then repeat changing order from “Our top picks” to “Review score”

Both should return the same items, regardless of their ordering

Implementation

- Run tests by instantiating each relation with specific test data
- Can use any standard unit testing framework

```
1 import static org.junit.Assert.*;
2 import org.junit.Test;
3
4 public class BookingSearchTest {
5
6     @Test
7     public void searchMetamorphicTest() {
8
9         // create query
10        BookingQueryObject query = new BookingQueryObject();
11        query.setDestination("Rome");
12        query.setCheckIn("11/12/2018");
13        query.setCheckOut("11/15/2018");
14        query.setAdults(2);
15
16        // Source test case
17        BookingSearchResult st = Booking.search(query);
18
19        // Follow-up test case
20        BudgetFilter filter = new BudgetFilter();
21        filter.setMinBudget(120);
22        filter.setMaxBudget(180);
23        filter.setCurrency("US");
24        BookingSearchResult fut = Booking.search(query, filter);
25
26        // Metamorphic relation assertion
27        assertTrue("Not a subset", fut.isSubset(st));
28    }
29 }
```

Figure 2. Sample implementation of a metamorphic test in JUnit 4.

Identification of metamorphic relations

- Input-driven approach
 - Think of changes to the program's inputs that should produce changes in the outputs
 - Based on data type (e.g. list – adding/removing/ordering and query – filtering, sorting)
- Output-driven approach
 - Starts from possible relations among outputs and then thinking about how changes to inputs lead to these outputs
 - Example for search include subsets, disjoint sets

Use in Compilers

Remove dead source code from a program

Should not alter the functionality

1. Run a program with inputs using profiler to identify executed code
2. Create new version by removing some dead code (statements not executed in step 1)
3. Run new program on same inputs, reporting any observed changes as a failure

33

Graphics Drivers

GraphicsFuzz: testing graphics drivers

1. Image is rendered using a shader program (shader compiler translates program into code for a gpu). Output image is the original image
2. Apply transformation to shader program that should not change original image (e.g. adding +0.0 to an arithmetic expression)
3. An image is rendered using the modified shader program.
4. Compare original and modified program

34

Artificial Intelligence

Testing of Simulated Autonomous Drones (Fraunhofer)

1. Define a model of a flying scenario and observe drone behavior in this scenario
2. Generate multiple variations of scenario where outcome should be the same (e.g. the drone should behave the same whether flying north or south if distances and relative positions are the same)

35

Artificial Intelligence

DeepTest

1. Testing tool for self-driving cars driving by Deep Neural Networks (DNNs)
2. Generates synthetic test cases with different driving conditions such as rain or fog where the car should behave in a similar way



Figure 4. A sample of erroneous behaviour found by DeepTest [19].

36

Metamorphic Testing Interactive Debuggers

Interactive Metamorphic Testing of Debuggers

Sandro Tolksdorf
sandro@sandrotolksdorf.de
TU Darmstadt, Germany

Daniel Lehmann
mail@dlhmann.eu
TU Darmstadt, Germany

Michael Pradel
michael@binaervarianz.de
TU Darmstadt, Germany

ABSTRACT

When improving their code, developers often turn to interactive

through *breakpoints*, to inspect the intermediate program state such as the values of local variables, and to closely follow the control-flow

37

Motivation

- **Interactive debuggers** are integral to software development
- Provide the ability to **add breakpoints**
- What if the **debugger itself is buggy**?
 - Perhaps a variable is missing/not shown in the debugger. Could assume the variable does not exist in program
 - One approach is differential testing

38

Motivation

1	▶ // 1. paused here	▶ // 1. paused here
2	// -> action: continue	// -> action: step over
3	foo();	foo();
4	function foo() {	function foo() {
5	▶ stmt; // 2. pauses here	▶ stmt; // 2. pauses here
6	} // because of breakpoint	} // as well

(a) Initial test run.

(b) Follow-up test run.

Figure 1: Example of a metamorphic test case for debuggers.

Both should stop at same point due to breakpoint

39

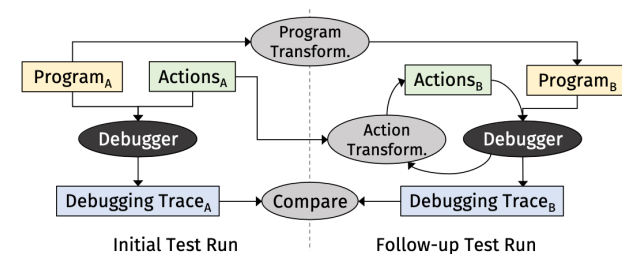


Figure 5: Overview of metamorphic testing for debuggers.

40

Table 1: Metamorphic transformations for debuggers.

Transformation	Short Description
<i>Action Transformations:</i>	
Add breakpoint and continue	Adding breakpoint at line l should have no influence other than additional pauses at l .
Replace continue by step	Control actions have a subset relation (continue $\subset$ step out $\subset$ step over $\subset$ step into) based on where they pause. Thus, sometimes, a continue can be replaced by a step.
Breakpoint sliding	Setting breakpoint at line l and sliding to l' should be equal to directly setting it at l' .
<i>Program Transformations: (Actions are transformed accordingly.)</i>	
Insert dead code	Insertion of statically dead code should have no influence on pauses or program state.
Remove dead code	Removal of dynamically dead code should have no influence.
Add parameter	Additional formal parameter should have no influence other than appearing as undefined in the function's scope.
Add no-operation	Insertion of $x = x$;, where x is in scope, should have no influence.
Integer literal to expression	Replacing n by $(n1+n2)$, where $n = n1 + n2$ should have no influence.

41

MT in Bioinformatics

42

Challenge – Data Driven Apps

Objective value 0.647004

Model iml_cdg_edit_correct

Media cdg_edit_correct_malt5

Simulation Information

Method Name: mtslp

Solution Type: OPTIMAL

Environmental Conditions: Env. Conditions(4)

Objective Function: max: R_BIOMASS_Ec_RL1515_VT_75p37M = 0.0

Biomass value: 0.0

Unit Conversions:

Metabolite ID	Metabolite Name	Value
M_R3_e	Iron (Fe3+)	1000.0
M_R20_e	D-glucose	10.0
M_R20_e	H2O H2O	10.0

Environment.from_compounds ["malt", "h2o", "o2", "pi", "nh4", "mn2", "zn2", "so4"]

FBA (iml1515_model1)

Objective: 0.7665324982307016

Status: Optimal

```

medium["EX_co2_e"] = 0
medium["EX_malt_e"] = 10
medium["EX_sel_e"] = 0
medium["EX_tungs_e"] = 0
medium["EX_slnt_e"] = 0

iml1515_model.medium = medium
iml1515_model.medium

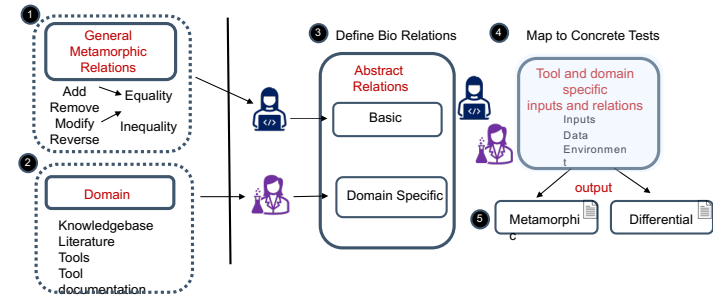
'EX_pi_e': 1000,
'EX_fe3_e': 1000,
'EX_o_e': 1000,
'EX_mn2_e': 100,
'EX_fe2_e': 1000,
'EX_co2_e': 1000,
'EX_mg2_e': 1000,
'EX_co2_e': 1000,
'EX_mn2_e': 1000,
'EX_cu2_e': 1000,
'EX_cobalt2_e': 1000,
'EX_h2o_e': 1000,
'EX_malt_e': 10,
'EX_mol_e': 1000,
'EX_so4_e': 1000,
'EX_nh4_e': 100,
'EX_o_e': 1000,
'EX_na_e': 1000,
'EX_cl_e': 1000,
'EX_o2_e': 1000

iml1515_model.slim_optimize()
1.7797995283262230

```

43

Types of Relations



Marsh et al. ICST 2025

44

Example Relations

Basic

Follow on Test	Check	Relation
Change carbon source	No. reactions	equality
Optimize non-biomass	biomass	inequality

Domain

Follow on Test	Check	Relation
Change carbon source	biomass	equality
Decrease glucose	biomass	inequality